

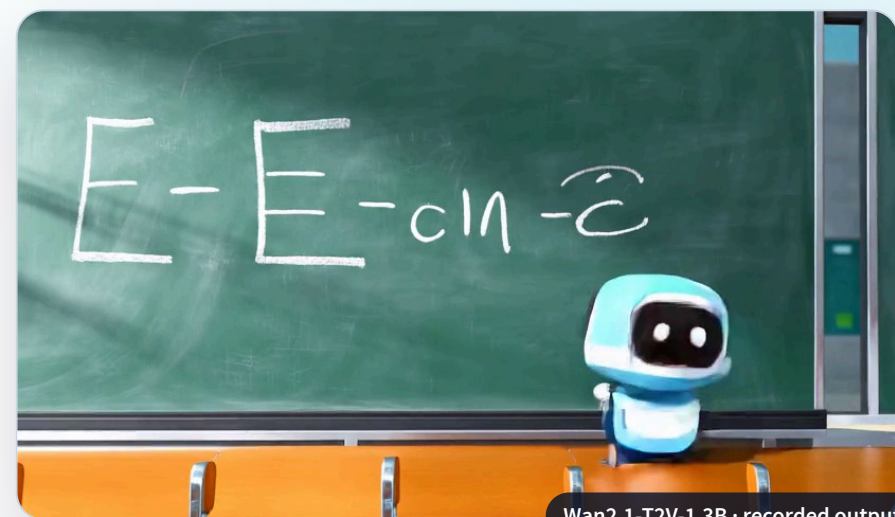
CS 486/686 • LECTURE 24

Dissecting Diffusion and World Models

From noise to video to an interactive world

[Yuntian Deng](#)

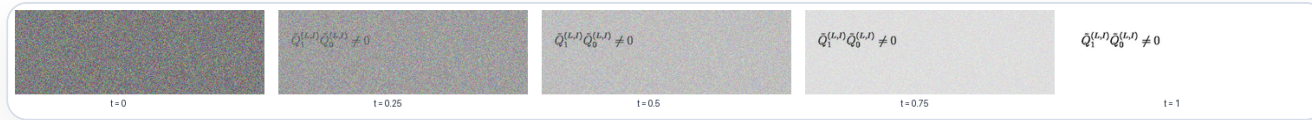
One model, end to end: Wan2.1-T2V-1.3B



Wan2.1-T2V-1.3B · recorded output

START WITH THE RESULT

An image appears from noise



Recorded model output.

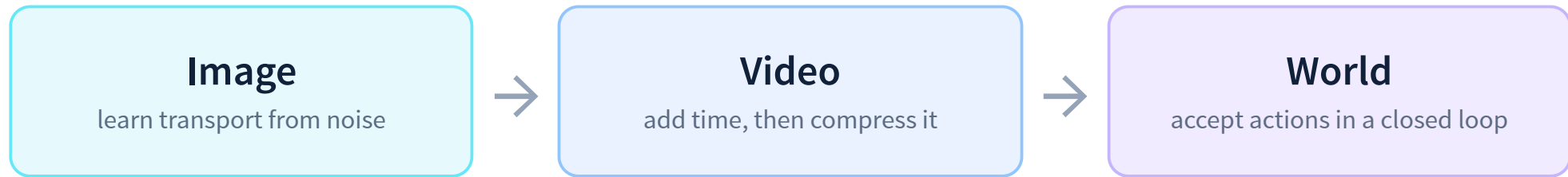
This is *markup2im*, an earlier DDPM-based equation generator—not Wan. We use it to expose the same big idea: start unstructured, refine repeatedly.

Today: preserve that intuition, then dissect the flow-matching video model Wan2.1.

Source: [Deng et al., “Latent Alignment and Variational Attention,” 2017.](#)

THE ENTIRE LECTURE IN ONE LINE

One image → one video → one interactive world



Explain flow matching as learned transport.

Trace every tensor through Wan2.1.

Separate the DiT from the VAE decoder.

Define what makes a generator a world model.

Begin with two endpoints of the same shape



z_{noise}
known distribution; easy to sample



$$\tilde{Q}_1^{(L,I)} \tilde{Q}_0^{(L,I)} \neq 0$$

z_{data}
an example from the data distribution

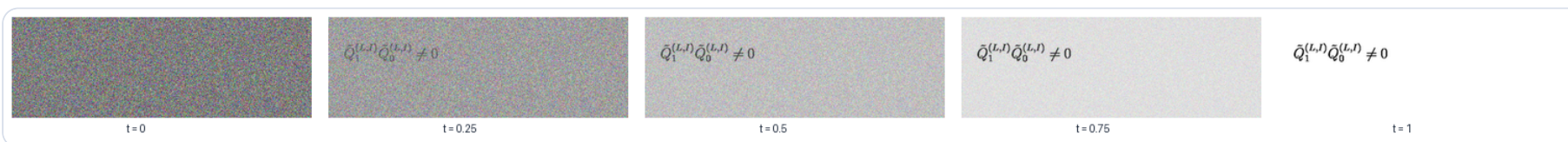
Generation asks for a path from a sample we can draw to a sample we want.

FIXED NOISE · FIXED DATA · MOVE TIME

A straight path connects the endpoints

INTERACTIVE

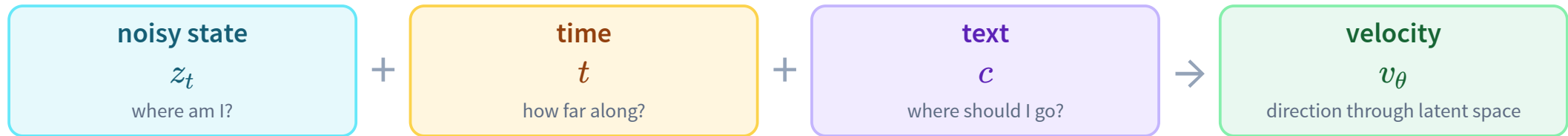
This pixel-space picture teaches the rule; Wan applies it to compressed video latents.



$$z_t = (1 - t)z_{\text{noise}} + tz_{\text{data}}$$

TRAINING TARGET

The model learns which way to move



$$v^* = z_{\text{data}} - z_{\text{noise}}, \quad \mathcal{L} = \|v_\theta(z_t, t, c) - v^*\|_2^2$$

The straight training path gives us the correct velocity for free.

INFERENCE

Generation integrates the learned velocity

- 1 Sample $z_0 \sim \mathcal{N}(0, I)$: begin at noise.
- 2 Ask $v_\theta(z_t, t, c)$ which direction the prompt suggests.
- 3 Take a numerical step: $z_{t+\Delta t} \approx z_t + \Delta t v_\theta(z_t, t, c)$.
- 4 Repeat until $t = 1$, then decode the final latent.

Training learns a vector field. A solver follows that field from noise to data.

CONNECT TO THE FAMILIAR PICTURE

Flow matching and DDPM share the loop—not the target

DDPM-style diffusion

Usually defines a stochastic noising process and trains an ϵ - or score-predictor.

Wan flow matching

Uses an interpolation path and trains a velocity predictor with `flow_prediction`.

Both generate by iteratively refining a noisy state.

Wan's released Diffusers scheduler uses UniPC with flow sigmas and `prediction_type="flow_prediction"`. See the [Flow Matching paper](#).

THE FIRST BRIDGE

An image is a one-frame video

[C, H, W]



[C, 1, H, W]

Same kind of values

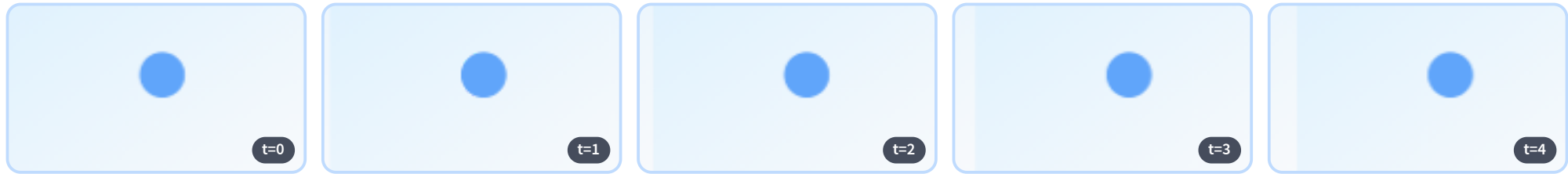
A video merely adds a time axis to the image tensor.

Checkpoint caveat

The architecture admits one frame; the official 1.3B release is a text-to-video checkpoint, not a separate 1.3B image model.

ACT II • ADD TIME

Video must make every frame agree



Appearance

The robot should remain the same robot.

Motion

Positions should change smoothly over time.

Physics

Objects should not pop in, vanish, or teleport.

WHY NOT MODEL PIXELS DIRECTLY?

Five seconds already means 97 million values

97,044,480

RGB values in $81 \times 480 \times 832 \times 3$

81 frames

About 5.06 seconds at 16 fps.

480×832

399,360 pixels per frame.

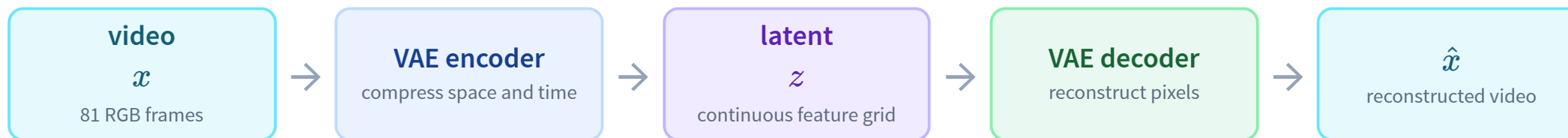
Every solver step

Repeated computation
would touch a huge tensor.

Compress first; run the expensive generative model in a smaller space.

COMPRESS PIXELS INTO LATENTS

The Wan-VAE is a learned video codec



The VAE is trained beforehand. Wan's DiT learns to generate its latent tensors.

A USEFUL ANALOGY—WITH A BOUNDARY

The VAE acts like a continuous video tokenizer

Text tokenizer

“robot waves” → integer IDs from a finite vocabulary.

[12984, 21143]

Wan-VAE encoder

81 frames → real-valued vectors on a space-time grid.

[-0.73, 1.20, ...]

Useful analogy: both shorten the sequence. Important difference: Wan has no codebook, token IDs, or vocabulary.

WAN-VAE SHAPE TRACE

Compress space by 8× and time by 4×

[3, 81, 480, 832]

RGB video



[16, 21, 60, 104]

continuous latent

time: 81 → 21

height: 480 → 60

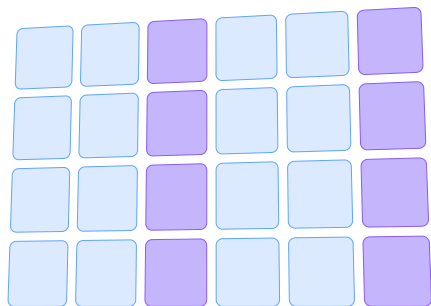
width: 832 → 104

channels: 3 → 16

Axis compression is $4 \times 8 \times 8$, but channels grow: overall, about **46× fewer scalar values**, not 256×.

TURN THE LATENT GRID INTO A SEQUENCE

The DiT sees 32,760 video tokens



Wan patch size: $(1, 2, 2)$ in
latent time, height, width.

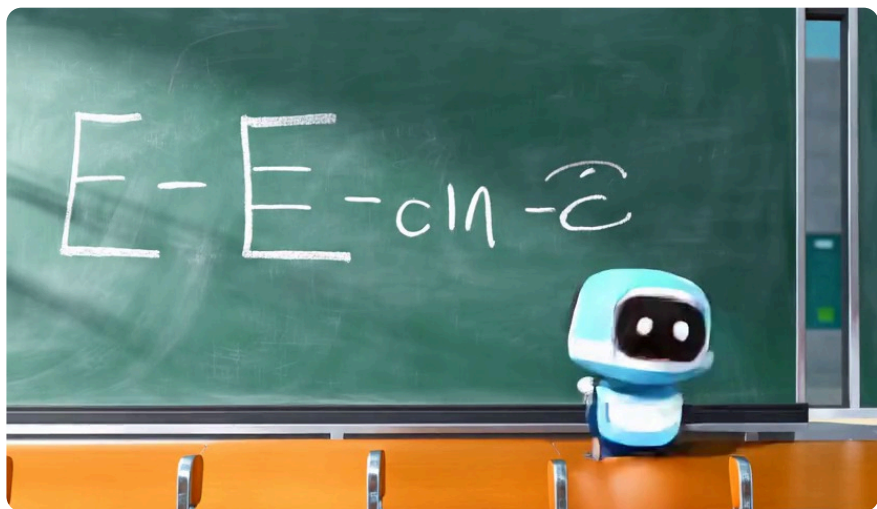
$$21 \times (60/2) \times (104/2) \\ = 21 \times 30 \times 52 = 32,760 \text{ tokens}$$

Each 64-value patch projects to width $d = 1536$.

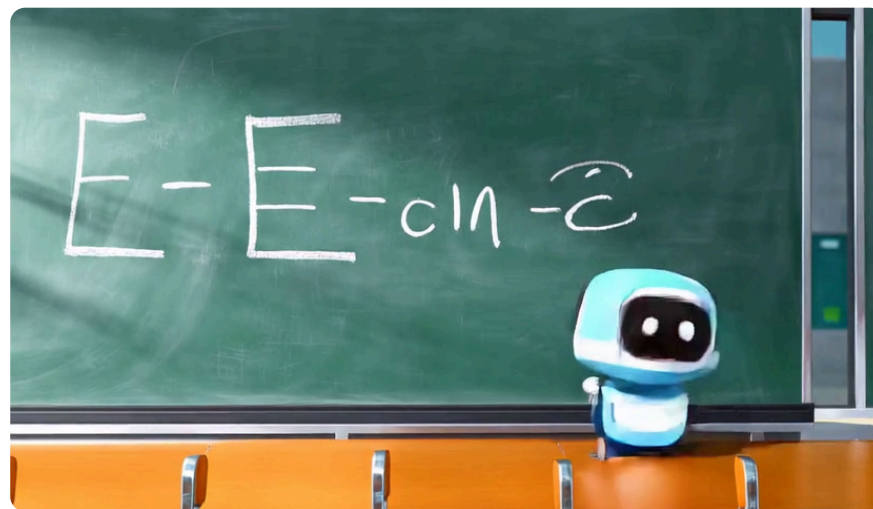
The transformer now handles a sequence of compact space-time patches—not 97 million pixel values.

COMPRESSION IS NOT FREE

The VAE preserves scenes better than tiny details



input frame
high-frequency text and edges



Wan-VAE reconstruction
same scene; some fine detail is softened

The DiT cannot generate information the codec systematically discards.

Real one-frame encode/decode through the official Wan-VAE; revision and hashes are in [wan_vae_roundtrip.json](#).

RETURN TO THE SAME EQUATION

Wan transports a video latent, not pixels

$$z_t = (1 - t)z_{\text{noise}} + tz_{\text{video}}, \quad z_t \in \mathbb{R}^{16 \times 21 \times 60 \times 104}$$

Same shape

Noise and clean video latents
line up element by element.

Same target

Predict velocity from noise
toward the clean latent.

Different scale

Every state represents
an entire 81-frame clip.

ACT III · DISSECT THE CHECKPOINT

Wan2.1-T2V-1.3B at a glance

Wan2.1 T2V-1.3B

latent flow-matching Diffusion Transformer

30

DiT blocks

1536

model width

12

attention heads

81

frames

16 fps

playback

480p

official target

“1.3B” names only the DiT. The full pipeline also loads a roughly 127M-parameter Wan-VAE and a much larger multi-billion-parameter umT5-XXL text encoder.

Sources: [Wan technical report](#), [official model card](#), and [Diffusers checkpoint configs](#).

THREE PRETRAINED COMPONENTS

Prompt in; video out

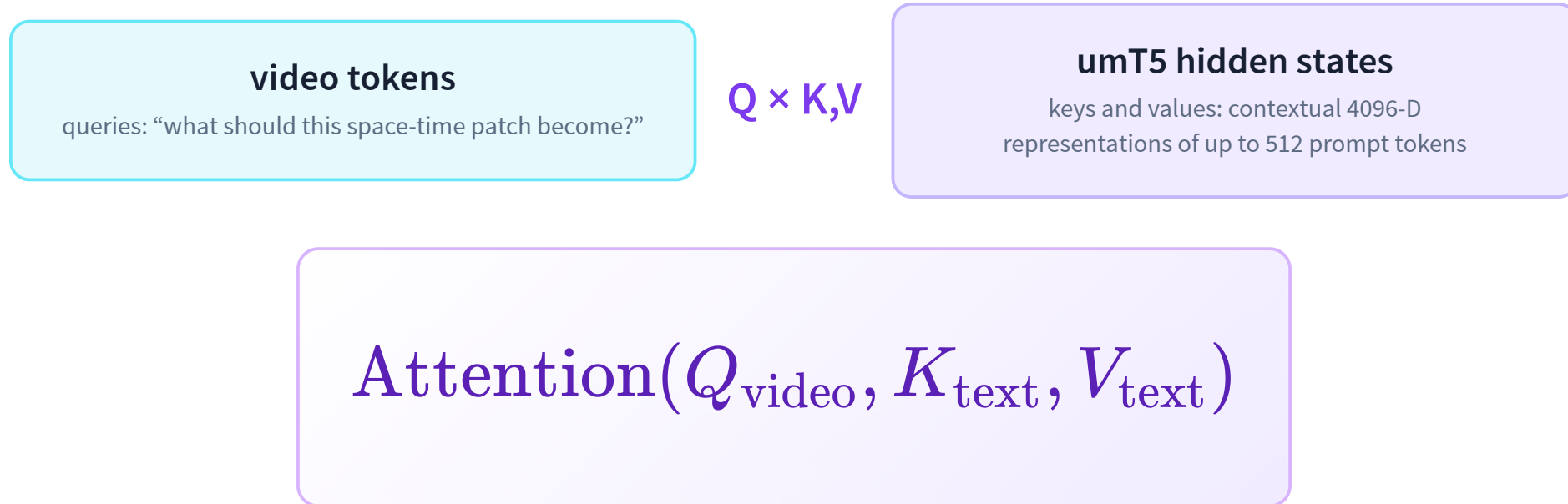


Latent noise is the evolving state; text conditions every DiT update through cross-attention.

The pipeline is not one network: language encoder, latent generator, and pixel decoder have different jobs.

LANGUAGE STEERS MOTION AND APPEARANCE

Text enters every DiT block through cross-attention



The DiT does not read raw words; it reads contextual hidden states from a text model.

INSIDE ONE OF 30 BLOCKS

A Wan block mixes video, text, and time



Self-attention coordinates space and time; cross-attention aligns that evolving clip with the prompt.

A NAME THAT OFTEN CAUSES CONFUSION

“Diffusion Transformer” does not mean pixel decoder

DiT: velocity predictor

Runs at every solver step. Reads a noisy latent and predicts how that latent should change.

`[16,21,60,104] → [16,21,60,104]`

Wan-VAE decoder: pixel renderer

Runs after transport is finished. Converts the final clean latent into 81 RGB frames.

`[16,21,60,104] → [3,81,480,832]`

The DiT changes latent velocity repeatedly; the VAE paints pixels once.

ONE TRAINING EXAMPLE

Freeze the encoders; train the DiT on velocity error

1 Encode a real video: $x \rightarrow z_{\text{data}}$. Encode its caption with umT5.

2 Sample $z_{\text{noise}} \sim \mathcal{N}(0, I)$ and $t \sim U[0, 1]$.

3 Mix them: $z_t = (1 - t)z_{\text{noise}} + tz_{\text{data}}$.

4 Train $v_{\theta}(z_t, t, c)$ toward $z_{\text{data}} - z_{\text{noise}}$.

Trainable: the 1.3B DiT. Frozen: Wan-VAE and umT5.

ONE GENERATED CLIP

UniPC follows prompt-guided velocities

$$\hat{v} = v_{\text{uncond}} + s (v_{\text{cond}} - v_{\text{uncond}})$$

Start

Sample one latent-noise video tensor.

Repeat 30 times

Run the DiT, combine conditional and unconditional velocities, then let UniPC update z .

Finish

Decode the final latent once and play 81 frames at 16 fps.

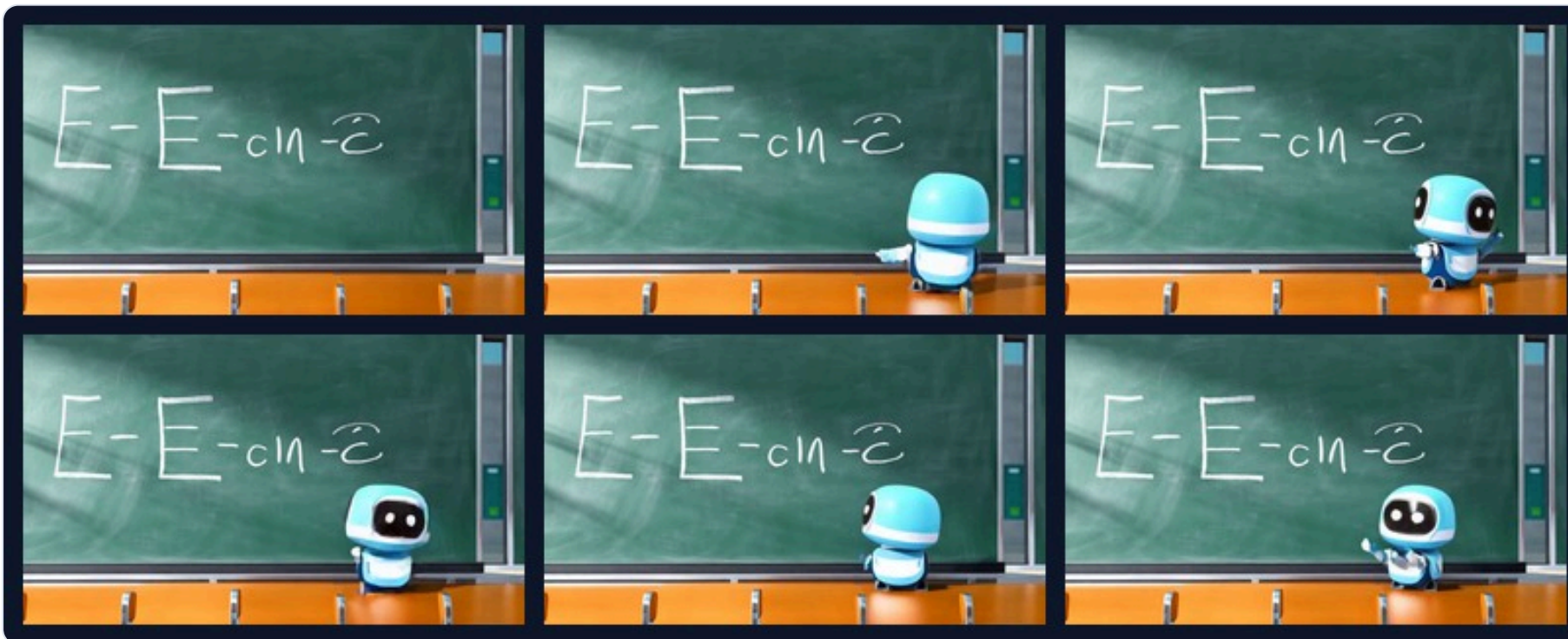
Flow matching still uses iterative inference; a straighter training path does not make generation one-shot.

OFFICIAL WAN CHECKPOINT · DETERMINISTIC RUN

Watch the generated clip

RECORDED MODEL OUTPUT

Prompt: a blue robot writes $E = mc^2$, turns, and waves · seed 486 · 30 steps · guidance 5.0



PDF filmstrip from the same 81-frame clip.

Generated with [Wan-AI/Wan2.1-T2V-1.3B-Diffusers](#); full revision, runtime, prompt, settings, and hashes are in [wan_samples.json](#).

NO HIDDEN SHAPE CHANGES

Trace every representation end to end

stage	representation	job
umT5	[≤512, 4096]	contextual prompt states
latent noise	[1, 16, 21, 60, 104]	state being transported
DiT tokens	[1, 32760, 1536]	attention sequence inside DiT
predicted velocity	[1, 16, 21, 60, 104]	same shape as latent state
decoded video	[1, 3, 81, 480, 832]	RGB output

Text stays a side input; the solver repeatedly changes only the video latent.

WHERE STOCK WAN STOPS

A beautiful clip is not yet an environment

Open loop

The prompt is fixed before generation begins.

Slow

Thirty large DiT evaluations are far from interactive latency.

Finite

The checkpoint emits a fixed 81-frame horizon.

Joint clip

Stock Wan denoises the whole bidirectional clip; its causal VAE does not make the DiT autoregressive.

To become a world model, the generator must observe a new action and predict what happens next—again and again.

ACT IV · CLOSE THE LOOP

A controllable video is still not a world model

Video generator

prompt once → fixed clip

$C \rightarrow O_1, \dots, O_{81}$

World model

observe → act → predict → feed back

$(o_{<t}, a_{\leq t}) \rightarrow o_t$

Control becomes world modeling only when actions arrive during generation and outputs become the next inputs.

THE AUTOREGRESSIVE ANALOGY

Predict the next observation, then reuse it



feed the prediction back into history ↺

$$p(o_{1:T} \mid a_{1:T}) = \prod_{t=1}^T p(o_t \mid o_{<t}, a_{\leq t})$$

Like next-token prediction, except each “token” can be a frame or a short latent video chunk.

HOW RESEARCHERS ADAPT WAN

Three changes turn open-loop video into a world model

1 • Add conditions

Encode current frame and history.
Embed mouse, keyboard, or game actions.

2 • Train closed loop

Predict causal chunks, not one fixed clip.
Include self-generated histories
to reduce exposure bias.

3 • Make it fast

Distill many solver steps into a few.
Stream chunks and cache reusable context.

The DiT can stay recognizable while its inputs, training distribution, and inference loop change.

Direct Wan2.1 example: [minWM](#).

EVIDENCE, NOT JUST A THOUGHT EXPERIMENT

Wan-family models have already become interactive

minWM

A direct Wan2.1 adaptation: action-conditioned next-chunk prediction, self-forcing, distillation, and streaming generation.

Matrix-Game 2.0

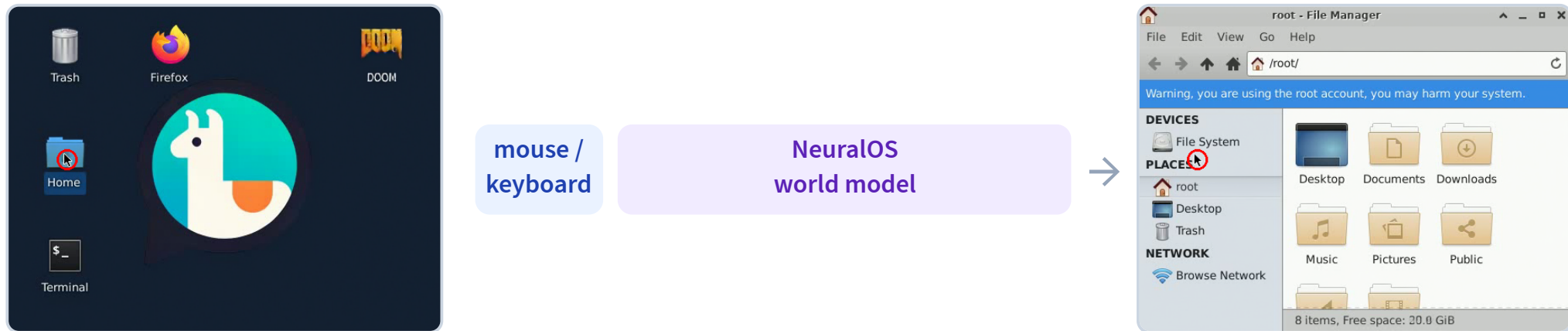
A Wan-architecture-lineage system that targets long, real-time, action-conditioned game simulation.

“Fine-tune Wan into a world model” is a concrete engineering recipe—not a claim that stock Wan is already one.

Sources: [minWM repository](#) and [Matrix-Game 2.0 project](#).

ACT V • COURSE-ENDING CASE STUDY

NeuralOS learns the next screen after your action

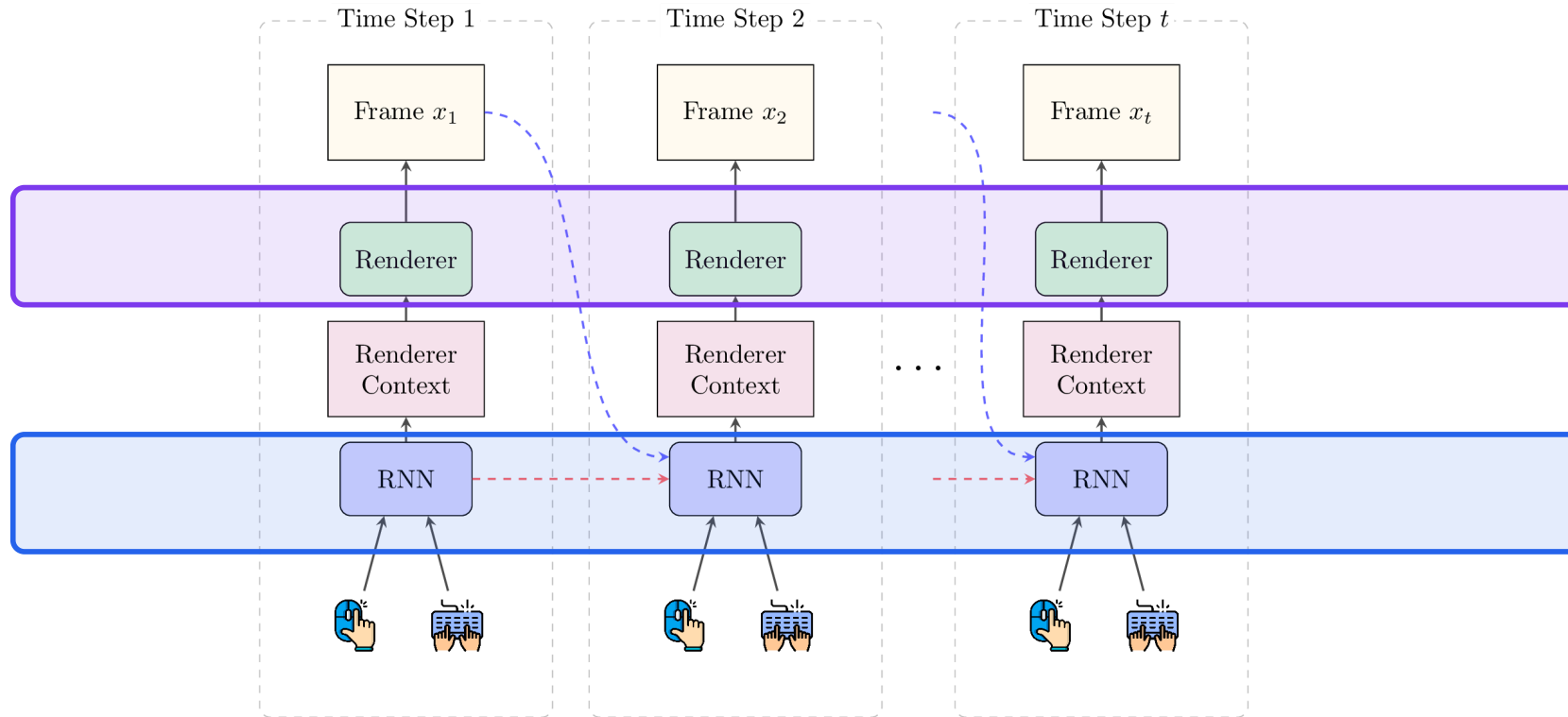


It is a learned GUI simulator—not an operating-system kernel, and not a Wan fine-tune.

Source: [Deng et al., “NeuralOS: Towards Simulating Operating Systems via Neural Generative Models,” 2025.](#)

A DIFFERENT ARCHITECTURE, THE SAME FORMULATION

State tracker first; visual renderer second



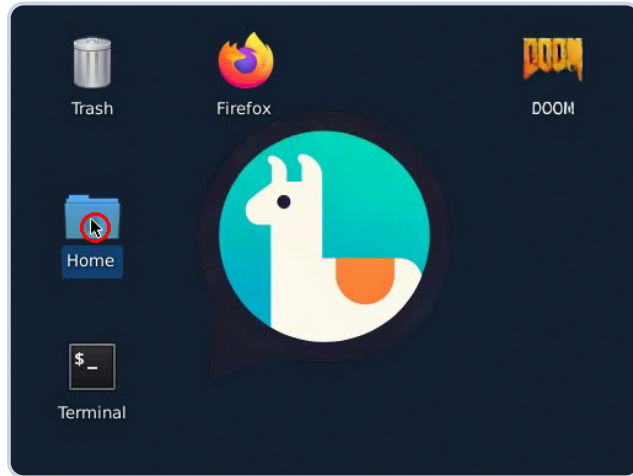
Blue: recurrent state remembers interaction history. **Violet:** latent diffusion renders the next screen.

ACTION-CONDITIONED ROLLOUT

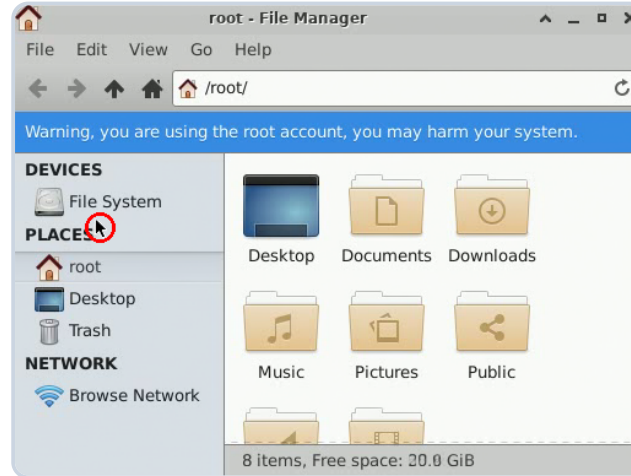
One user action changes what comes next

RECORDED MODEL OUTPUT

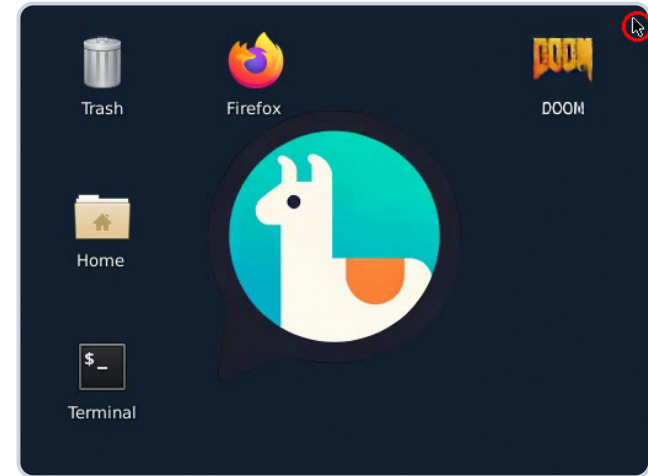
Step through: double-click Home → file manager opens → close it.



1 • double-click Home
action enters the model



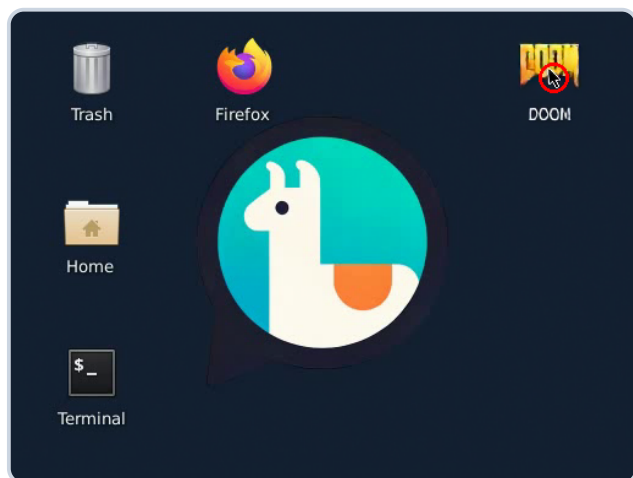
2 • file manager opens
predicted transition



3 • close → desktop
prediction fed back

WHAT CAN A LEARNED ENVIRONMENT ABSORB?

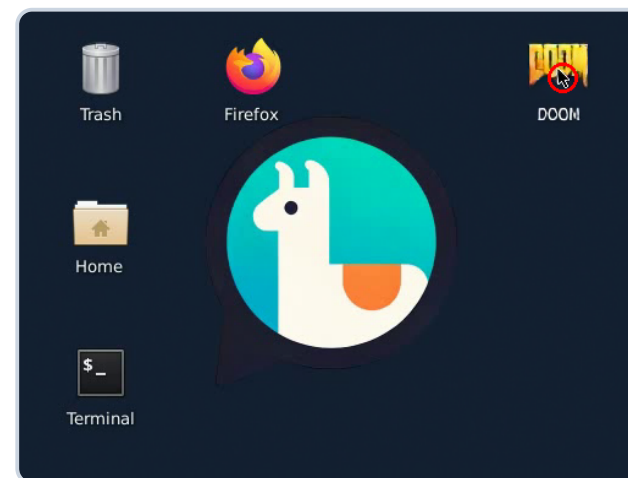
Synthetic demonstrations can add a new behavior



1 • Doom icon appears
synthetic demonstrations



2 • double-click launches
learned transition

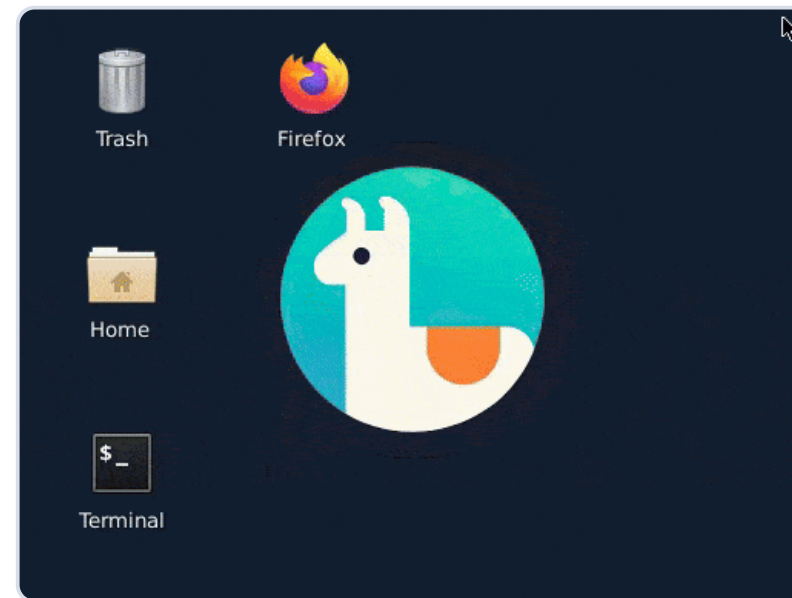
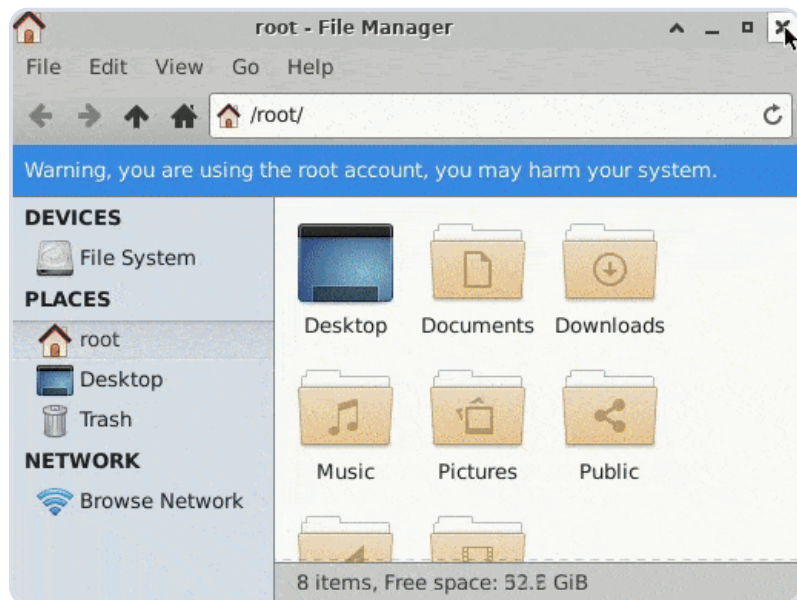


3 • ESC returns
action-conditioned response

Demonstrate a transition—even synthetically—and the learned simulator can make it part of its world.

NeuralOS result; the Doom trajectories were fabricated and inserted into training data.

The model can mistake correlation for action



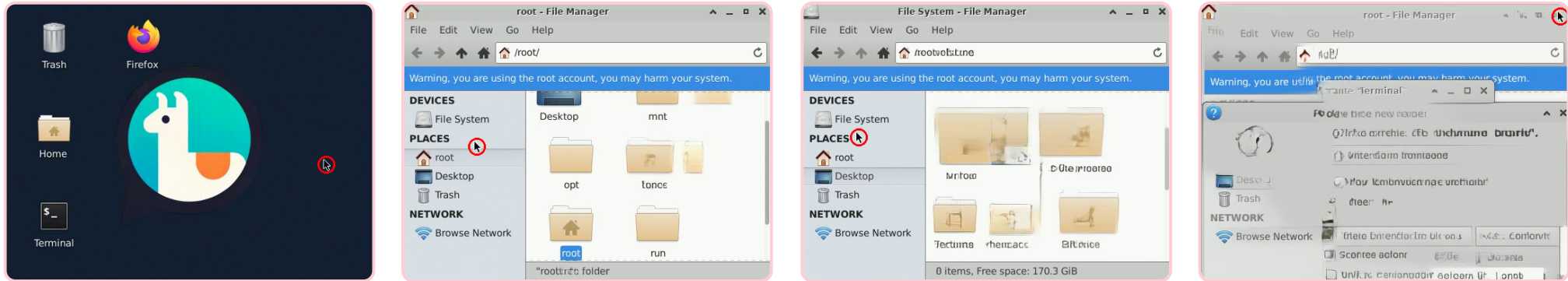
Training shortcut

The agent almost always clicked after hovering near x, so the model learned “hover means close.”

Data fix

Random exploration supplies hover-without-click counterexamples and separates correlation from action.

Small errors become the next inputs



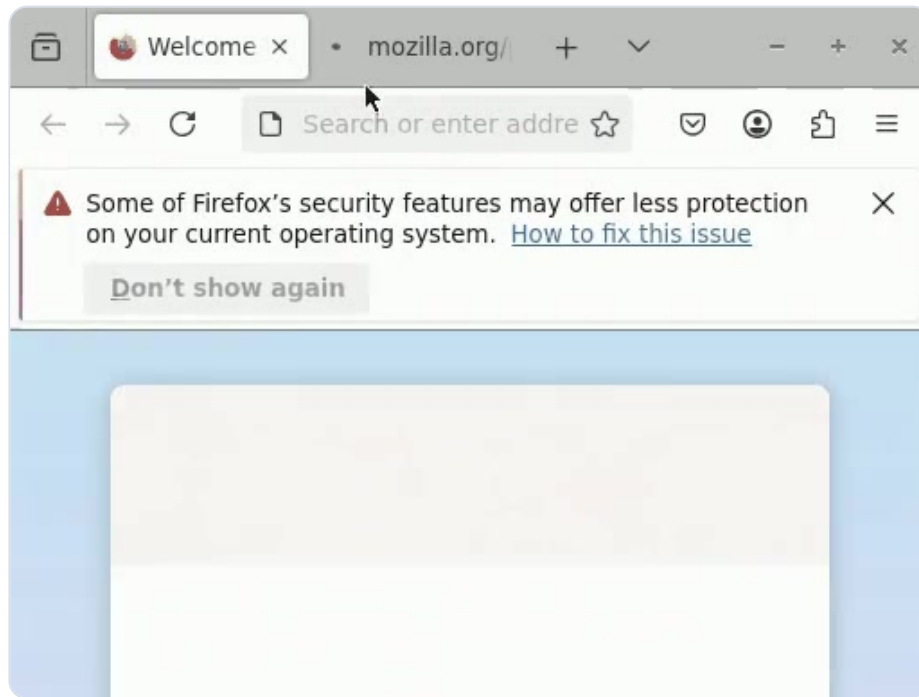
one imperfect frame → model conditions on it → a larger error → drift

Train on self-generated histories too. Scheduled sampling and self-rollout training narrow the train–inference gap.

END WHERE AN ENVIRONMENT BEGINS

A learned operating system

RECORDED + LIVE LINK



Recorded NeuralOS rollout.



[Try neural-os.com](https://neural-os.com) ↗

A generator becomes an environment when it predicts what happens next, responds to actions, and feeds its own output back.

Wan2.1 and NeuralOS use different architectures. The shared idea is action-conditioned prediction in a closed loop.